

Table of Contents

- Working on OWL
 - Nodes overview
 - Partitions
 - SLURM
 - Interactive login
 - Job submission
 - Further useful SLURM commands

Previous topic

[Software modules](#)

Next topic

[Storage system](#)

This Page

[Show Source](#)

Quick search

Working on OWL

Nodes overview

Node types on OWL								
Node name	#	CPU type	Cores/ CPUs	Frequency (GHz)	L3 (MB)	Cache	Memory (GB)	Infini-band
owl01amd	1	AMD EPYC 7452	64/ 128	2.35	16		1007	EDR
owl02intel	1	Intel Xeon E7-8857 v2	48/ 48	3.0	30		256	EDR
genius 01-12	12	AMD EPYC 9354	64/128	3.25	256		384	HDR
milan 01-12	12	AMD EPYC 7343	32/ 64	3.2	256		251	HDR
fat 01-02	2	Intel Xeon E5-4650 v3	48/ 96	2.1	30		1024	EDR
haswell 01-12	12	Intel Xeon E5-2650 v3	20/ 40	2.3	24		128	EDR
ccgp 01-04	4	Intel Xeon Gold 6248	40/ 80	2.5	28		176	EDR
cclake 01-04	4	Intel Xeon Gold 6248	40/ 80	2.5	28		176	EDR
rogp 01-04	4	AMD Epyc Rome 7452	64/ 128	2.35	16		240	EDR
rome 01-04	4	AMD Epyc Rome 7452	64/ 64	2.35	16		240	EDR
sdl 01	1	AMD Epyc 9254	48	2.9	256		384	HDR

Nodes ccgp01-04 are equipped with 1 Nvidia Tesla V100 GPU card each. Nodes rogp01-04 are equipped with 1 RADEON INSTINCT MI50 GPU card each. Node sdl01 is equipped with a Nvidia H100 GPU card.

owl01amd is the AMD login node to compile code for AMD architectures also for pre- and postprocessing. owl02intel is the Intel login node to compile code for Intel architectures also for pre- and postprocessing. (Demanding pre- and postprocessing steps should be pushed to compute nodes).

Partitions

The above compute and logon nodes are grouped into different partitions (queues) in SLURM:

Partition	Hostnames	Nodes	Max nodes per job	Max walltime [h]	QOS	notes
fat	fat01-02	1	1	3-00:00		
genius	genius01-12	12	12	3-00:00		
haswell	haswell01-12	12	12	3-00:00		
milan	milan01-12	12	12	3-00:00		
cclake	cclake01-04,	4	4	3-00:00		
rome	rome01-04,	4	4	3-00:00		
ccgp	ccgp01-04,	4	4	3-00:00	nvgpu	Nvidia Tesla V100 GPU
rogp	rogp01-04,	4	4	3-00:00		AMD INSTINCT GPU RADEON MI50
sockdolager	sdl01,	1	1	3-00:00	sdlgpu	NVIDIA GPU H100

SLURM

All acces to the compute nodes is via the [SLURM](#) workload manager. There are two ways to access the nodes

- 1. interactively by opening a shell and
- 2. passively by submitting jobs to the scheduling system.

Interactive login

To interactively acces a compute node the [salloc](#) command is used:

```
# reserve a single process on a compute node of a certain partition
salloc -p [partition] -N 1 -n 1

# make an exclusive allocation, so that the resource is not shared with other users
salloc -p [partition] -N 1 -n 1 --exclusive
```

`-N` or `--nodes` gives the number of requested nodes, `-n` or `--task-per-node` the number of CPUs requested per node. On systems with hyperthreading, the number of CPUs is different to the number of cores, i.e. one physical core is split into two virtual CPUs. Therefore, to use complete physical cores for your task, you need to provide `--cpus-per-task=2` in your `salloc` command. If you don't specify a runtime of your job with `-t` or `--time`, it will default to 8 hours. A typical allocation command would therefore be:

```
# allocate all cpus on a node of the cclake partition for 1 day
salloc -p cclake -N 1 --tasks-per-node=64 --cpus-per-task=2 --time=1-00:00
```

You will be automatically forwarded to the node. To stop the allocation, type `exit` or press `CTRL-d`.

Job submission

Jobs are submitted to OWL with SLURM using the `sbatch` command, typically in combination with a submit script.

A slurm submission file: `submit.wrf`

```
#!/bin/csh

#SBATCH -p milan
#SBATCH --nodes=2
#SBATCH --tasks-per-node=32
#SBATCH --cpus-per-task=2
#SBATCH -J "my famous job"
#SBATCH --mail-user=mymail@kit.edu
#SBATCH --mail-type=all
#SBATCH -t 0-01:00:00

set rundir = /bg/data/myproject/myrundir

module purge

module add openmpi/3.1.6-intel-2021.7.1-cuda wrf

cd ${rundir}
rm rsl*
srun wrf.exe
set exitcode = $?
cd -

exit ${exitcode}
```

This submission file can be submitted to the SLURM workload manager with `sbatch`:

```
# submit batch script to SLURM
sbatch submit.wrf
```

After submission, a `jobid` will be returned. The `jobid` is required to either cancel a running job `scancel [jobid]` or to query information about a job with `scontrol show jobid [jobid]`. The list of currently running jobs can be obtained with `squeue` which can be also filtered for a specific user with `-u [username]`.

Further useful SLURM commands

```
# get an overview of the HPC resources, partitions, free nodes
sinfo

# for a specific partition
sinfo -p [partition]

# get the list of unavailable nodes
sinfo -R

# check the supposed starting time of a job
squeue --start

# provide a priority list for the partition (it will use the one that
# becomes free first)
sbatch -p [partition1],[partition2]

# change the partition of a queued job
scontrol update jobid=[jobid] partition=[new partition]

# job dependencies, will start only if job [jobid] finished successfully
sbatch -d afterok:[jobid]
```

SLURM is quite powerful. It can do almost everything you can imagine. Sometimes it's worth the effort to read the docs.